Programming Assignment 5: Implementing Sets

Due: Thursday, March 5th at 11pm CT

Learning Goals

- To practice implementing data structures, such as a `Set`
- To utilize `ArrayLists` and `Iterators`
- To implement sorting and searching algorithms by modifying pre-existing algorithms
- To develop code that makes use of the object-oriented design principles of encapsulation, inheritance, and polymorphism

Assignment Guidelines and Specifications

This is a **pair programming assignment**, which means you can work with a partner on this assignment. You are not required to work with a partner and are welcome to complete the assignment individually if you wish. If you choose to work with a partner, there are some important rules you must follow:

- You should follow these best practices when pair programming.
- Your partner **must be in the same section** as you and have the same TA.
- You will write **one** program file and include both of your names in the file header. **Only one member of the pair will submit the code to Gradescope.** After submitting the assignment, on the submission page, click "View or edit group" on the upper righthand corner of the screen. Enter the name of the other partner in order to add them to the submission. After doing this, the other partner should get an email confirming that they were added to the submission. **If you do not add your partner to the Gradescope submission, the other member of the pair will not get any credit for completing this assignment.** After adding the other partner, check that both partners are able to see the submission on their Gradescope accounts. This will mean that you have both been correctly linked to the submission.
- If you choose to use late days, both partners must have the required amount of late days. If either partner does not, **both** partners will receive a 0 on this assignment.

Introduction

In this assignment, you will implement three set classes (`AbstractSet`, `UnsortedSet`, and `SortedSet`) that implement the `ISet` interface. This interface models a set and specifies the methods that set data structures should have. As a reminder, the elements in a set have no definite order and are all unique.

Getting Started

Here are the necessary files for this assignment:

- **`ISet.java`**: This is the interface that all the other set classes implement. You should not edit this code and do not need to submit this file.
- **`AbstractSet.java`**: This is an abstract class (skeletal) implementation of the set interface. You will need to implement as many methods as possible according to the given specifications and submit this file.
- **`UnsortedSet.java`**: This is an extension of `AbstractSet` where the elements of the set are in no particular order. You will need to complete this file according to the given specifications and submit this file.
- **`SortedSet.java`**: This is an extension of `AbstractSet` where the elements of the set are in sorted ascending order. You will need to complete this file according to the given specifications and submit this file.
- **`SetTester.java`**: This file contains the main method used for testing and the provided correctness tests. You will need to write your own tests in this file and submit this file.
- **`Stopwatch.java`**: This file contains the Stopwatch class, which you will need to use to measure the time elapsed. You should not edit this code and do not need to submit this file.

AbstractSet

This class is a skeletal implementation of the `ISet` interface (like the `AbstractCollection` or `AbstractList` classes in the Java standard library). In this class, you should implement as many methods as possible in terms of other methods, regardless of the efficiency. Realize there are some methods that cannot be implemented in `AbstractSet`. Part of the work you need to do in this assignment is figuring out which methods can and cannot be implemented in `AbstractSet`.

Here are some notes on what is **not** allowed in this class:

- This class may not have any instance variables.
- You may not add any methods to this class that are not part of the `ISet` interface or the `Object` class. You are only to implement methods from the `ISet` interface.
- A critical design principle of abstract classes is that they should not rely on or assume any subclass implementation details. In other words, `AbstractSet` should not make *explicit* references to `SortedSet`, `UnsortedSet`, or `ArrayList`. You may be making *implicit* references to call some methods such as `iterator()`. If the implicit calls weren't allowed, you couldn't get anything done in `AbstractSet`. (If you're interested, you can read more about this specific design principle, the dependency inversion principle.)

You might be wondering: *How can I implement methods in AbstractSet when there isn't an internal storage container or other instance variables?* By using other methods in the class!

For example, we can use the `iterator()` method to implement `contains()` within `AbstractSet`. You won't be able to implement the `iterator()` method in the `AbstractSet` class since you need an internal storage container to do that. So, within the `contains()` method, you will be calling another method that will be implemented later. Here is an example of what the `contains()` method could look like:

```
public boolean contains(E item) {
    Iterator<E> it = this.iterator(); // get an iterator for this set.
    // do a linear search using the iterator object
    while (it.hasNext()) {
        if (it.next().equals(item)) {
            return true; // found it. Done!
        }
    }
    return false; // never found item
}
```

Additionally, you can take advantage of the fact that `ISet` extends the `Iterable` interface. This means we can streamline the code using a for-each loop:

```
public boolean contains(E obj) {
    for (E val : this) {
        if (val.equals(obj)) {
            return true;
        }
    }
    return false;
}
```

The starter code includes a `toString()` method as another example of how to use an iterator. (You should use this method to print out your set objects during testing.) You can assume the `Iterator` class implements the `remove()` method. Note that you can also use other methods besides `iterator()`. For example, `containsAll()` could make use of `contains()`.

Implement **one** of the `intersection()`, `union()`, or `difference()` methods. **Explain in the comment at the top of `SetTester` why it would be unwise to implement all three of `intersection()`, `union()`, and `difference()` in the `AbstractSet` class.**

You should implement the `equals()` method in the `AbstractSet` class and will need to override it in only `SortedSet`. This is because two objects that are both `ISets` are equal if they contain the same elements. The status of sorted or unsorted (and the order of the elements) does not matter when checking for equality in `AbstractSet`. This means it is possible that an `UnsortedSet` will equal a `SortedSet`. (This convention is used in much of the Java standard library.)

UnsortedSet

The `UnsortedSet` class implements a set (in which there are no duplicate elements) that is unsorted from the client's point of view. This means that the elements in this set are not maintained in any particular order. This class extends the `AbstractSet` class. In this class:

- You must use an `ArrayList<E>` as the internal storage container.
- Implement the methods from `ISet` that you did not implement in `AbstractSet`.
- Override methods from `AbstractSet` if you can implement them more efficiently now. See the section below on target Big O's for the different methods.
- For methods that return an `ISet` such as `union()`, `intersection()`, and `difference()`, return an instance of `UnsortedSet`.

These are the Big O's you should be able to achieve for the methods in `UnsortedSet`. Assume there are already N elements in `UnsortedSet`. For methods with another set, assume there are N elements in that set as well. If an implemented method in `AbstractSet` already achieves the target Big O, there is no need to override the method.

- `add()` $\rightarrow O(N)$
- `addAll()` $\rightarrow O(N^2)$
- `clear()` $\rightarrow O(N)$
- `contains()` $\rightarrow O(N)$
- `containsAll()` $\rightarrow O(N^2)$ (worst case is this set does contain all of the elements of the other set and each set contains N elements)
- `difference()` $\rightarrow O(N^2)$
- `equals()` $\rightarrow O(N^2)$ (worst case, assuming the sets are equal)
- `intersection()` $\rightarrow O(N^2)$
- `iterator()` $\rightarrow O(1)$ (Recall, your internal storage container is an `ArrayList` and `ArrayList` already has a method to get an `Iterator`)
- `remove()` $\rightarrow O(N)$
- `size()` $\rightarrow O(1)$
- `union()` $\rightarrow O(N^2)$

For each method, state the achieved Big O of the method in a comment at the top. For some of the methods, you will have to determine the Big O of methods from the `ArrayList` class. Refer to the `ArrayList` documentation and think about how we implemented methods in our simple array-based list class (`GenericList`) to determine the Big O of methods from `ArrayList`.

SortedSet

The `SortedSet` class implements a set (in which there are no duplicate elements) that maintains the elements in sorted order. Given that, the generic data type E must be a class that implements the `Comparable` interface. The class header for `SortedSet` modifies E to guarantee this. This class extends the `AbstractSet` class. In this class:

- You must use an `ArrayList<E>` as the internal storage container.
- Implement the methods from `ISet` that you did not implement in `AbstractSet`.
- Override methods from `AbstractSet` if you can implement them more efficiently now. See the section below on target Big O's for the different methods.
- For methods that return an `ISet` such as `union()`, `intersection()`, and `difference()`, return an instance of `SortedSet`.

Additionally, you must complete a constructor to build a `SortedSet` out of an `ISet<E>`. This will be in addition to the default constructor. The constructor is as follows:

```
public SortedSet(ISet<E> otherSet)
```

- Construct a `SortedSet` object out of `otherSet`.
- Precondition(s): `otherSet != null`
- Postcondition(s): `otherSet` is not altered as a result of calling this operation.

You must also complete two additional methods that return the min and max values in a `SortedSet`:

```
public int min()
```

- Precondition(s): none
- Returns: minimum value in this set.

```
public int max()
```

- Precondition(s): none
- Returns: maximum value in this set.

These are the Big O's you should be able to achieve for the methods in `SortedSet`. Assume there are already N elements in `SortedSet`. For methods with another set, assume there are N

elements in that set as well. For methods that involve two sets, **these target Big O's only apply if the other set is also a `SortedSet`.** If the other set is not a `SortedSet`, the target Big O is the same as for the `UnsortedSet` class. If an implemented method in `AbstractSet` already achieves the target Big O, there is no need to override the method.

- `SortedSet(ISet<E> otherSet)` $\rightarrow O(N * logN)$
- `add()` $\rightarrow O(N)$
- `addAll()` $\rightarrow O(N)$ (if the other set is also a `SortedSet`, use an algorithm like merge from mergesort)
- `clear()` $\rightarrow O(N)$ (possible with a single statement)
- `contains()` $\rightarrow O(logN)$
- `containsAll()` $\rightarrow O(N)$ (if the other set is also a `SortedSet`, worst case is this set does contain all of the elements of the other set and each set contains N elements)
- `difference()` $\rightarrow O(N)$ (if the other set is also a `SortedSet`)
- `equals()` $\rightarrow O(N)$ (worst case is if the other set is a `SortedSet`, assuming the sets are equal)
- `intersection()` $\rightarrow O(N)$ (if the other set is also a `SortedSet`)
- `iterator()` $\rightarrow O(1)$ (Recall, your internal storage container is an `ArrayList` and `ArrayList` already has a method to get an `Iterator`)
- `remove()` $\rightarrow O(N)$
- `size()` $\rightarrow O(1)$
- `union()` $\rightarrow O(N)$ (if the other set is also a `SortedSet`)

For each method, state the achieved Big O of the method in a comment at the top. For some of the methods, you will have to determine the Big O of methods from the `ArrayList` class. Refer to the `ArrayList` documentation and think about how we implemented methods in our simple array-based list class (`GenericList`) to determine the Big O of methods from `ArrayList`.

When writing the methods, take advantage of the fact that the elements in `SortedSet` are in sorted order. For example, consider the `contains()` method. This can be implemented in $O(N)$ time in `AbstractSet` using the `iterator()` method. However, in `SortedSet`, we can use binary search to cut down the runtime to $O(logN)$. We highly recommend you watch the posted video on binary search (which can be found on Panopto Video on Canvas) and refer to the posted slides (which can be found here).

You can speed up other methods where the `ISet<E> otheSet` parameter is a `SortedSet`. For example, the `intersection()` method will look something like the following example. If the `otherSet` is not an instance of `SortedSet`, use the constructor that converts it to a `SortedSet`.

```java
public ISet<E> intersection(ISet<E> otherSet) {
    // we know E is a Comparable, but we don't know if
    // otherSet is a SortedSet.

    // local var of correct type so we only have to cast once
    SortedSet<E> otherSortedSet;

    // do we need to create a new SortedSet based on otherSet or is
otherSet really a SortedSet?
    if (!(otherSet instanceof SortedSet<?>)) {
        otherSortedSet = new SortedSet<>(otherSet); // O(NlogN)
    } else {
        otherSortedSet = (SortedSet<E>) otherSet;
    }

    SortedSet<E> result = new SortedSet<E>();

    // fill result with modified merge algorithm, should be O(N)
    return result;
}
```

For the sorting and searching algorithms, you may **not** use any of the sorting or binary searching methods from the Java standard library. You may use the built-in linear search methods such as `ArrayList` `contains()` and `indexOf()`.

We highly recommend that you watch the three posted videos on simple sorting algorithms and more advanced sorting algorithms (which can be found on Panopto Video on Canvas). Additionally, we recommend that you reference the posted slides (which can be found here for the simple sorting algorithms and here for the more advanced sorting algorithms). You will not be able to use the sorting and searching code from the slides exactly as they are given. Instead, you should adapt them to your classes since your internal storage containers are `ArrayLists`, not native arrays. If you use the code from the slides, **you must cite the source with a comment.**

AbstractSet, UnsortedSet, SortedSet Methods

In all the classes, these same methods must be implemented. The only exceptions are certain methods in `AbstractSet` that are impossible to implement given it is an abstract class. Additionally, some `AbstractSet` implementations may be reused in classes that extend it if the efficiency cannot be improved upon.

1. Implement the following method:

```
public boolean add(E item)
```

   - Add `item` to the set.
   - Precondition(s): `item != null`
   - Returns: `true` if this set is altered as a result of this operation, `false` otherwise.

2. Implement the following method:

```
public boolean addAll(ISet<E> otherSet)
```

   - Add all elements of `otherSet` that are not already present in this set to this set.
   - Precondition(s): `otherSet != null`
   - Returns: `true` if this set is altered as a result of this operation, `false` otherwise.

3. Implement the following method:

```
public void clear()
```

   - Make this set empty.
   - Precondition(s): none
   - Postcondition(s): `size() == 0`

4. Implement the following method:

```
public boolean contains(E item)
```

   - Check if `item` is in this set.
   - Precondition(s): `item != null`
   - Returns: `true` if `item` is in this set, `false` otherwise.

5. Implement the following method:

```
public boolean containsAll(ISet<E> otherSet)
```

   - Check if all elements in `otherSet` are in this set.
   - Precondition(s): `otherSet != null`
   - Returns: `true` if all elements of `otherSet` are in this set, `false` otherwise.

6. Implement the following method:

```
public ISet<E> difference(ISet<E> otherSet)
```

   - Compute the difference between this set and `otherSet` (the elements that are in this set but not in `otherSet`). Neither set shall be altered as a result of this operation.

- Precondition(s): `otherSet != null`
- Returns: a new set that is the difference between this set and `otherSet`.

7. Implement the following method:

```
public ISet<E> intersection(ISet<E> otherSet)
```

- Compute the intersection between this set and `otherSet` (the elements that are in this set and in `otherSet`). Neither set shall be altered as a result of this operation.
- Precondition(s): `otherSet != null`
- Returns: a new set that is the intersection between this set and `otherSet`.

8. Implement the following method:

```
public ISet<E> union(ISet<E> otherSet)
```

- Compute the union between this set and `otherSet` (the elements that are in this set or in `otherSet`). Neither set shall be altered as a result of this operation.
- Precondition(s): `otherSet != null`
- Returns: a new set that is the union between this set and `otherSet`.

9. Implement the following method:

```
public boolean equals(Object other)
```

- Determine if this set is equal to `other`. Two sets are equal if they have exactly the same elements, regardless of order.
- Precondition(s): none
- Returns: `true` if `other` is an instance of `ISet` and has the same elements as this set, `false` otherwise.

10. Implement the following method:

```
public Iterator<E> iterator()
```

- Precondition(s): none
- Returns: An `Iterator` object for the elements of this set.

11. Implement the following method:

```
Public boolean remove(E item)
```

- Remove `item` from set, if it is present.
- Precondition(s): `item != null`
- Returns: `true` if this set is altered as a result of this operation, `false` otherwise.

12. Implement the following method:

```
public int size()
```

- Precondition(s): none
- Returns: the number of items in this set.

Tips

- As mentioned before, implement as many methods in `AbstractSet` as possible **regardless of efficiency**. Clever usage of `iterator()` can implement most methods.
- Reuse existing code and methods. Keep in mind that certain operations are equivalent. For such operations, it is only necessary to implement one of them and reuse that implementation in the other operation. *(Hint: pay attention to the method headers in `ISet.java`)*.
- We highly recommend that you follow the given structure of `intersection()` for `SortedSet` methods that have an `otherSet` parameter. This significantly cuts down on the code complexity by funneling all possible `ISet` implementations into their `SortedSet` equivalent. The alternative would be to write separate code for each `ISet` implementation (imagine if we had 10 other `ISet` implementations – code complexity would increase drastically).
- Be mindful of casting. In methods where the parameter is an `ISet<E>`, we know that `otherSet` must contain elements of the same type as the calling set. For equals(), we cannot guarantee that the given object is a set containing the same type of elements. For such cases, utilize the wildcard operator.

Testing

In `SetTester.java`, you are required to write at least 1 test per method per class that implements that method. In other words, if there is a method in `AbstractSet` and you do not override it in `UnsortedSet` or `SortedSet`, you just need to write one test for that method. On the other hand, if there is a method in `AbstractSet` that you do override in both `UnsortedSet` and `SortedSet` you have to write code for all the versions).

Therefore, you will need to have at least 1 test covering all implementations of:

```
add(), addAll(), clear(), contains(), containsAll(), difference(),
intersection(), union(), equals(), remove(), size(), min()*, max()*
```

Note that `iterator()` works similarly to a constructor, and thus does not need to be explicitly tested (it should be implicitly tested by methods that use `iterator()`).

**min() and max() are exclusive to SortedSet.*

In previous assignment guides, we shared a few notes with you about writing test cases. Those notes are copied here for your convenience:

- You may copy the structure of the provided tests.
- Any test cases that you write should not violate the preconditions of the method.
- You must group the test cases by method and comment which method the test case is for.
- You are not required to write test cases for any private helper methods or constructors.
- **Delete the test cases that we have provided for you.**

Experiments

When your classes are completed, run the method `largeTest()` in the `SetTester` class. This method has you pick a file and then adds all of the words in the file to various Sets. The method uses the `SortedSet`, the `UnsortedSet`, the Java `HashSet`, and Java `TreeSet`. The time is displayed for the operation to execute. Test this method with a small file at first to ensure it works. Then, test on files of various sizes (Project Gutenberg is a great way to find some).

1. Using data from at least four different files, report your results in a comment at the top of the `SetTester` class. Your results should have the following statistics.
    a. File name
    b. File size
    c. Total words
    d. Factor increase in total words compared to the previous row
    e. Total unique words
    f. Factor increase in unique words compared to previous row
    g. Actual time
    h. Factor increase in time compared to previous row

Include the statistics above for each data structure tested (`UnsortedSet`, `SortedSet`, `HashSet`, `TreeSet`) in separate tables. Here's an example showing the first three rows of such a table for the `Unsorted Set`:

File	Size (kb)	Total Words	Increase from prev row (total words)	Unique Words	inc from prev row (unique words)	Actual time	inc from prev row (actual time)
Foo.txt	67	1218	-	271	-	0.052 sec	-
Bar.txt	151	2109	1.7x	483	1.8x	0.24 sec	4.6x
Baz.txt	517	7927	3.8x	1702	3.5x	1.16 sec	4.8x

Along with the timing experiment above, answer the following questions. Include your answers in a comment at the top of the `SetTester` class.

2. What do you think the order (Big O) of the two `processText()` methods are for each kind of Set? Assume N = total number of words in a file and M = number of distinct words in the file. M = the size of the set when finished. Explain your answer in 1 to 2 sentences. Make sure to reference the timing data in your explanation.

3. What are the orders (Big O) of your add methods? What do you think the Big O of the `HashSet` and `TreeSet` add methods are? Explain your answers. Make sure to reference the timing data in your explanations.
4. What are the differences between `HashSet` and `TreeSet` when printing out their contents?

Submission Checklist

Before you submit, run through our submission checklist to make sure you didn't forget anything! Did you remember to:

1. Complete all the required classes (`AbstractSet, UnsortedSet, SortedSet`)?
2. Not add any instance variables to AbstractSet?
3. Not add methods not declared in `ISet` to `AbstractSet`?
4. Not reference `UnsortedSet`, `SortedSet`, or `ArrayLists` in `AbstractSet`?
5. Implement as many methods as you can in `AbstractSet` regardless of efficiency?
6. Override methods from `AbstractSet` in `UnsortedSet` and `SortedSet` if a more efficient version can be implemented?
7. State the Big O of all your methods in `UnsortedSet` and `SortedSet`?
8. Pass the tests in `SetTester.java` and delete those tests?
9. Add your own tests to `SetTester.java`?
10. Complete the experiments and place your answers in a comment at the top of the `SetTester.java`, along with the question about implementing intersection, union, and difference in `AbstractSet`?
11. Fill in the header comment in all submitted files?
12. Check the preconditions in public methods and throw exceptions as necessary?
13. Follow the program hygiene guide?
14. Follow the assignment specifications from page 3 of the Assignment handout?

If you have done all these things, then you're ready to submit `SetTester.java`, `AbstractSet.java`, `UnsortedSet.java`, and `SortedSet.java` to Gradescope!

Thank you to Professor Mike Scott for creating the previous version of this assignment!